

PhishGuard AI

Machine Learning Based Phishing Website Detector

A Semester Project Combining Machine Learning, Backend Development, and Web Integration

Project Report

Submitted By

M.Arsalan (CU-5041-2024)
M.Shehriyar (CU-5076-2024)
M.Wasi (CU-5067-2024)
Zayed Ayub (CU-5106-2024)

Supervised By

Mr. Jalal Khan

Class Code

BS-CS-B/A -2024

Date

23/07/2026

1. Overview

PhishGuard AI takes a URL as input, validates it, extracts a defined set of features from it, and feeds those features into a trained machine learning model. The model returns a prediction indicating whether the URL is likely to be a phishing site or a legitimate one. The result is displayed back to the user along with the model's confidence and a breakdown of the extracted features.

The project consists of:

- A Flask backend that handles URL validation, feature extraction, and prediction
- An HTML/CSS/JavaScript frontend for user interaction
- A feature extraction module
- A predictor module
- A URL validator
- Two machine learning models trained in Google Colab

2. Selected Features

The models were trained using exactly the following 11 features extracted from a URL:

#	Feature
1	having_IP_Address
2	URL_Length
3	having_At_Symbol
4	double_slash_redirecting
5	Prefix_Suffix
6	having_Sub_Domain
7	SSLfinal_State
8	HTTPS_token
9	Shortining_Service
10	DNSRecord
11	URL_of_Anchor

No features outside this list are used by the models.

3. Machine Learning Models

Two classification models are included in the project:

- XGBoost
- LightGBM

Both models were trained on the same dataset and using the same 11 features. The user can select which model to use for prediction directly from the web interface.

4. Dataset

Attribute	Value
File	Training Dataset.arff
Total Samples	11,055
Number of Features	11
Target Variable	Result
Classes	Legitimate, Phishing
Train/Test Split	80 / 20

5. Technologies Used

- Python
- Flask
- HTML
- CSS
- JavaScript
- XGBoost

- LightGBM
- Scikit-learn
- Pandas
- NumPy
- BeautifulSoup
- Requests
- Joblib
- Git
- GitHub
- Google Colab

6. Folder Structure

```

PhishGuard-AI/
|-- app.py                # Flask application entry point
|-- requirements.txt      # Python dependencies
|-- models/              # Serialized, trained ML models
|   |-- xgboost_model.pkl
|   +-- lightgbm_model.pkl
|-- utils/               # Core backend logic
|   |-- feature_extractor.py
|   |-- validator.py
|   +-- predictor.py
|-- templates/           # HTML templates rendered by Flask
|   |-- index.html
|   +-- about.html
|-- static/              # Frontend assets
|   |-- css/
|   +-- js/
+-- dataset/
    +-- Training Dataset.arff

```

templates/ — Contains the HTML pages served by Flask, including the home page and the about page.

static/ — Contains the CSS and JavaScript files that control the styling and client-side behavior of the front-end.

utils/ — Contains the backend logic modules responsible for URL validation, feature extraction, and prediction.

models/ — Contains the trained XGBoost and LightGBM models, saved using Joblib, which are loaded by the backend at runtime.

7. Backend Modules

7.1 validator.py

Responsible for validating the format of the URL entered by the user and confirming DNS existence before any feature extraction takes place. This acts as the first checkpoint in the pipeline, preventing malformed or non-resolvable URLs from being processed further.

7.2 feature_extractor.py

Responsible for extracting the 11 selected features from a given URL. This module inspects the URL's structure and, where required, its DNS records to compute each feature value used by the models.

7.3 predictor.py

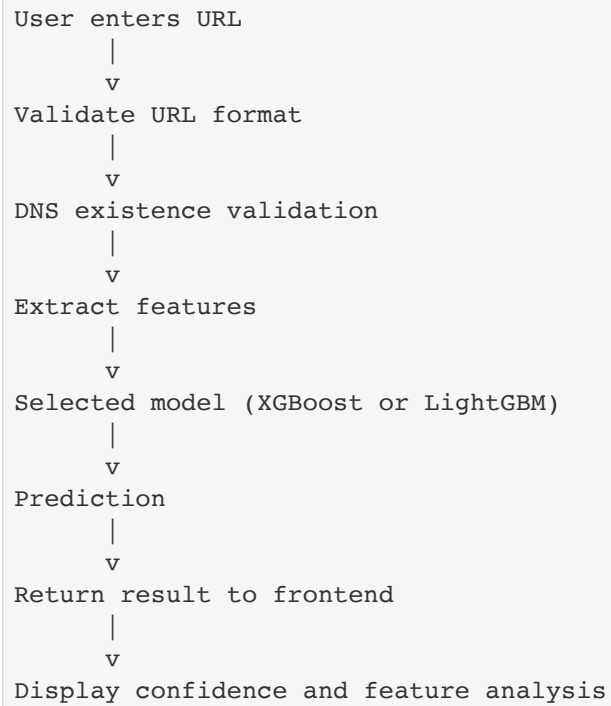
Responsible for loading the selected trained model (XGBoost or LightGBM) and generating a prediction based on the extracted feature vector. Returns the predicted class along with the model's confidence.

7.4 app.py

The Flask application entry point. Handles incoming requests from the frontend, coordinates calls to the validator, feature extractor, and predictor modules, and returns the final prediction result to the UI.

8. Prediction Workflow

The complete prediction pipeline follows these steps:



9. User Interface

The frontend includes:

- Home page — main entry point for scanning a URL
- About page — information about the project
- URL scanner — input field for submitting a URL for analysis
- Model selection dropdown — lets the user choose between XGBoost and LightGBM
- Prediction card — displays the final result (Legitimate or Phishing)
- Confidence percentage — shows the model's confidence in the prediction
- Feature analysis section — displays the values of the extracted features for the submitted URL

10. Installation

10.1 Clone the repository

```
git clone https://github.com/your-username/phishguard-ai.git
cd phishguard-ai
```

10.2 Create a virtual environment

```
python -m venv venv
```

Activate it:

On Windows:

```
venv\Scripts\activate
```

On macOS/Linux:

```
source venv/bin/activate
```

10.3 Install the required dependencies

```
pip install -r requirements.txt
```

10.4 Run the Flask application

```
python app.py
```

10.5 Open the application in your browser

```
http://127.0.0.1:5000
```

11. Example Usage

11.1 Example 1 — Legitimate URL

Input:

```
https://google.com
```

The validator confirms the URL format and DNS existence. Features such as `having_At_Symbol`, `Prefix_Suffix`, and `SSLfinal_State` are computed and passed to the selected model, which returns a Legitimate prediction with an associated confidence score.

11.2 Example 2 — Suspicious URL

Input:

```
https://google.com@evil.com
```

This URL contains an `@` symbol, which is flagged by the `having_At_Symbol` feature — a common technique used to disguise the true destination domain (`evil.com`) behind a trusted-looking prefix (`google.com`). Depending on the combined feature values, this increases the likelihood of a Phishing classification.

12. Limitations

- Only 11 features are used for classification
- Detection is based solely on URL structure
- No analysis of actual webpage content is performed
- No WHOIS-based domain age or ownership analysis
- No integration with real-time threat intelligence feeds
- DNS validation only confirms that a domain resolves, not that it is safe
- The system cannot guarantee 100% accuracy

Phishing detection in this project is probabilistic in nature. A prediction reflects the model's confidence based on learned patterns in the training data, not a definitive guarantee of a URL's legitimacy.

13. Future Improvements

- Incorporate additional URL and domain-based features
- Add WHOIS-based domain age and registration analysis
- Analyze webpage content in addition to URL structure
- Add typosquatting detection
- Improve confidence calibration of model outputs
- Retrain models on newer, larger phishing datasets
- Expose predictions through a dedicated API
- Deploy the application to a cloud platform

14. Conclusion

PhishGuard AI is a semester project that demonstrates the practical application of machine learning to a real-world security problem: phishing URL detection. It brings together machine learning, web development, backend development, feature engineering, model deployment, and software integration into a single working system.

The project does not claim to be a production-grade security tool. It reflects a specific, bounded implementation — 11 URL-based features, two trained models, and a Flask-based web interface — built to demonstrate the end-to-end process of taking a machine learning model from training to a usable application.